ELTE IK - Computer Science BSc

Final Exam Topics

9. Object oriented design

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Object oriented design

Development phases of large systems, development methods. SOLID design principles. Architectural patterns (MV, MVC, etc.). The role of design patterns, their classification (creational, structural, behavioral), and the presentation of 2 notable design patterns per category.

1 Development phases of large systems, their relationships

1.1 Development phases

1. Antecedent of solving the problem

   Before solving a problem we have to examine its feasibility and the manner of it. Result: *Feasibility study*, which answers the following:

   - Resources (hardware, software, specialist)
   - Costs
   - Deadline
   - Operation

2. Requirements description

   We usually produce it iteratively, and use the prototype for refinement (Figure 1).

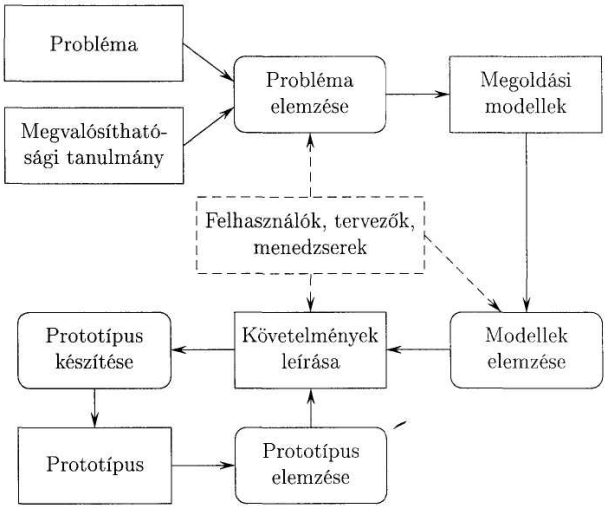

Figure 1: The process of producing the requirements description

Content of the requirements description:

- Problem
- Limiting factors (hardware, software, etc.)
- Acceptable solution

Types of requirements description:

- Functional requirements

  The description of the services and mappings of the system:

  - Form of starting
  - Input data (and the form of giving them)
  - Precondition of use, restrictions
  - The response to initiating a service, results
  - The form of appearance of the response
  - The relation between the input data and the response
- Non-functional requirements

  The non-functional requirements are usually sorted into three classes: product requirements, management requirements, external requirements. The classes can be further broken down (Figure 2).

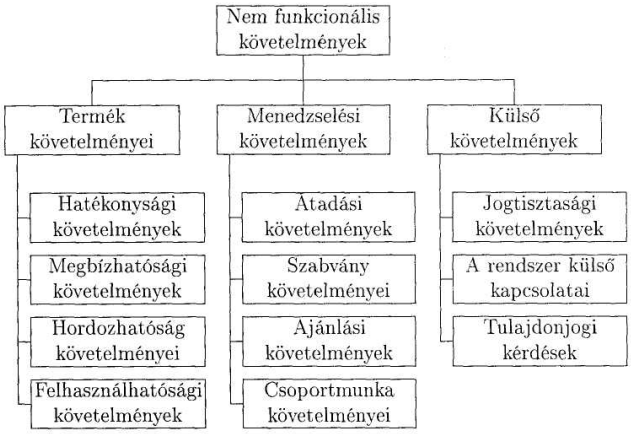

Figure 2: Classification of non-functional requirements

3. Analysis of requirements and prototype

   The following have to be examined:

   - Is the requirements description good in itself?
     - Consistent (no contradiction)
     - Complete
   - Validation examination

     (Does it conform to the problem imagined by the user?)
   - Feasibility examination

     (Is a solution conforming to the requirements feasible?)
   - Testability examination

     (Are the requirements formulated so that they are testable?)
   - Examination of the criteria of openness.

     (Do the requirements not contradict the requirement of modifiability, further developability?)

One tool of analyzing the requirements is prototype-making. The prototype is a solution of the problem created in a high-level programming environment, correct from the point of view of external behavior.

4. Program specification

   The program specification has to answer the following questions based on the requirements description:

   - What is the input data? (Form, meaning, appearance.)
   - What are the results? (Form, meaning, appearance.)
   - What is the relation between the input data and the result data?

5. Design

   During design we give the answers to the following questions:

   (a) Static model
   - Structure of the system
   - Program units, their task and connection

   (b) Dynamic model
   - How does the system solve the problem?
   - What units cooperate?
   - What messages play out?
   - States of the system and units
   - Events (under the effect of which a state change occurs)

   (c) Functional model
   - Through what data flows are the services realized?
   - What mappings play a role in the data flows?
   - What are the recommendations for the implementation?
     - Recommendation regarding the implementation strategy.
     - Prescription, recommendation regarding the programming language.
     - Recommendation regarding the testing strategy.

   In practice two design methods have become widespread: *procedural* and *object-oriented*

   (*procedural*: we start from the functions, operations to be realized, and based on these we break the system into smaller components, modules
   object-oriented: instead of the functions of the system, we put the data at the center of the design. The data used by the system will, in a certain sense, correspond to the objects.)

6. Implementation

   Important considerations:

   - Representation (representation of data)
   - Realization of events, mappings
     Algorithms and optimizations

   One fundamental question of implementation is the implementation style. A few important elements of good programming style:

   - application of different levels of abstraction
   - use of the inheritance technique, hierarchical system of abstraction levels
   - breaking into abstraction levels within a class (declaration + realization)

- limited visibility;
- information hiding;
- encapsulation.

7. Verification, validation

   Does the system satisfy the expectations imposed on it?

   Verification: proof of correctness according to the specification

   Validation: fulfillment of quality prescriptions (robustness, efficiency, resource requirement)

   The process of this: testing, whose phases are:

   - Unit test
   - System test

   Testing can be done in two ways:

   - black box - Only the discovery of the errors themselves
   - white box - Discovery of the location of the errors

8. System follow-up and maintenance

   Maintenance: software-type work that becomes necessary after deployment [e.g.: fixing hidden errors, adaptation work (new hardware/software environment), further development work]

   System follow-up: the management-type, documentation tasks of keeping in contact with the users [e.g.: registration of configurations, management of versions, management of documentation]

9. Documentation

   A large-sized program in itself cannot be regarded as a software product without documentation. A good documentation is built up as follows.

   - User description
     - Task description
     - Runtime environment
     - Developments, versions
     - Installation
     - Usage
     - Authors
   - Developer description
     - Modules (and their structure)
     - Classes (and their connection)
     - Dynamic behavior of the system
     - Implementation of the classes (data structures, template classes)
     - Testing

1.2 Relationships of the development phases

To describe the development phases we can use several kinds of models

1. Waterfall model

   The individual phases follow each other; the modifications happen knowing the run results. A modification performed in a certain phase also affects all subsequent phases.

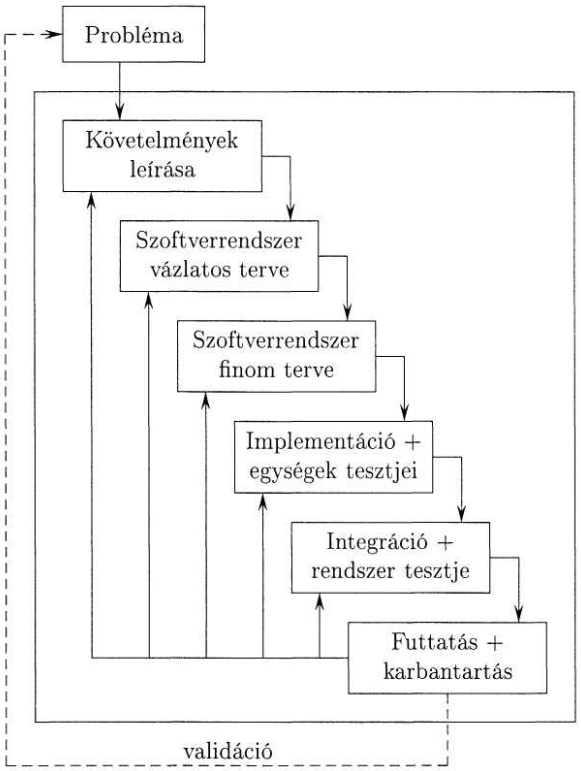

Figure 3: Waterfall model

   Its disadvantages:

   - A new service requires modification in every phase
   - Validation can require the repetition of the entire life cycle

2. Evolutionary model

   We produce a sequence of approximating versions, prototypes of the solution one after another, and thus proceed step by step all the way to the final solution. During this, when producing a version, the specification, the development, and the validation happen in parallel.

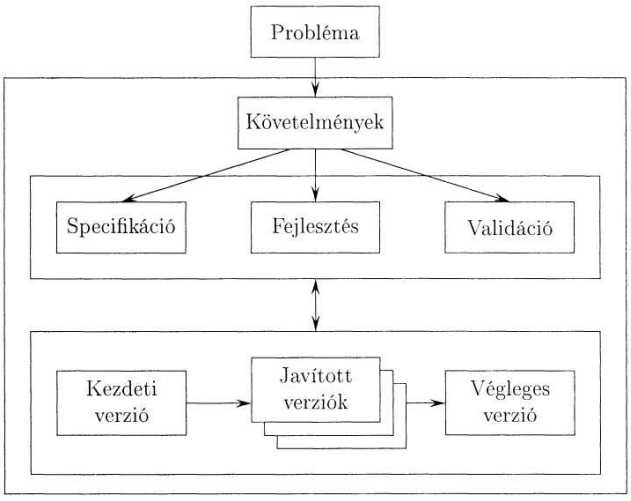

Figure 4: Evolutionary model

Its disadvantages:

- The project is hard to oversee
- Rapid development usually comes at the expense of documentedness.

3. Boehm's spiral model

   This model is an iteration model. The iteration can be modeled with one phase of the spiral, which can be broken into four sections:

   (a) Defining goals, paths, alternatives, restrictions

   (b) Risk analysis, strategy development

   (c) Solving the task, validation

   (d) Planning the next iteration

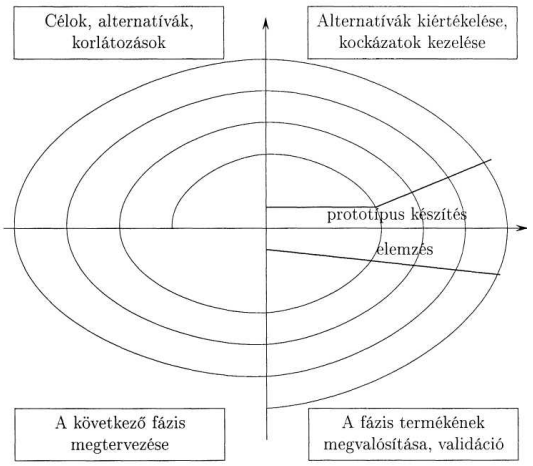

Figure 5: Spiral model

Its disadvantages:

- Applying the model is generally a labor-intensive, complex task.
- It is not easy to economically employ the specialists needed for elaborating the project.

2 SOLID design principles

In object-oriented programming SOLID is an acronym composed of the initial letters of the five basic design principles (Single responsibility principle, Open/closed principle, Liskov substitution principle, Interface segregation principle, Dependency inversion principle), and its goal is to make software design even more understandable, flexible, and maintainable. SOLID has nothing to do with the General Responsibility Assignment Software Patterns (GRASP). The father of the S.O.L.I.D. principles is Robert Cecil Martin, an American software engineer and consultant, who is not only the spokesperson of the "clean code movement", but also, among other things, one of the original formulators of the Agile Manifesto. These principles form the core of agile software development or adaptive software development. He presented the theory in his book Design Principles and Design Patterns, but as an acronym it only later became widespread through Michael Feathers.

- **Single Responsibility Principle** A class or module should have one, and only one, responsibility (that is: it should have one reason to change).
- **Open/Closed Principle** A class or module should be open for extension but closed for modification.

- **Liskov substitution principle** Every class should be replaceable with its descendant class without the correct operation of the program changing.
- **Interface segregation principle** The principle of separating interfaces (connecting surfaces): no client should be forced to depend on procedures that it does not even use.
- **Dependency inversion principle** High-level modules should not depend on low-level modules. Both should depend on abstractions.

3 Architectural patterns (MV, MVC, etc.)

TODO

3.1 MVC

The model-view-controller (MVC) is a software design pattern used in software design. In complex computer applications presenting a lot of data to the user, it is a frequent developer wish to separate the things belonging to the data (model) and to the user interface (view), so that the user interface does not influence the data handling, and so that the data can be reorganized without changing the user interface. The model-view-controller achieves this by separating the access to the data and the business logic from the presentation of the data and the user interaction, through the introduction of an intermediate component, the controller.

Traditionally used for desktop user interfaces, but nowadays it has also become popular for web applications. Popular programming languages such as JavaScript, Python, Ruby, PHP, Java already have MVC frameworks, without the need for separate installation, for developing web and mobile applications.

Breaking an application into several layers is common: presentation (user interface), domain logic, and data access. In MVC the presentation is further broken into view and controller. MVC determines the structure of an application much more than is characteristic of a design pattern.

Model

The domain-specific representation of the information handled by the application. The domain logic gives meaning to the bare data (e.g. it computes whether today is the user's birthday, or the total, taxes, and shipping costs for the items of a shopping cart).

Many applications use persistent storage procedures (such as a database) for storing data. MVC does not mention the data access layer separately, because it includes it in the model.

View

It presents the model in an appropriate form suitable for user interaction, typically in the image of a user interface element. Different views can exist for different purposes for the same model.

Controller

It processes and responds to events, typically user actions, and can also trigger changes in the model.[8]

MVC is often seen in web applications, where the view is the current HTML page, and the controller is the code that gathers the dynamic data and creates the content in the HTML. Finally, the model is represented by the content, which is usually stored in a database or in XML files.

Although MVC has many interpretations, the flow of control generally works as follows:[9]

- The user exerts some effect on the user interface (e.g. presses a button).
- The controller takes over the incoming event from the user interface, often through a registered event handler or callback.
- The controller establishes connection with the model, possibly updating it in a manner corresponding to the user's activity (e.g. the controller updates the user's cart). Complex controllers are often designed according to the command pattern, for the encapsulation of operations and to simplify extension.

- The view (indirectly) creates an appropriate user interface based on the model (e.g. the view creates the screen listing the contents of the cart). The view obtains its data from the model. The model has no direct knowledge of the view.
- The user interface waits for a new event, which starts the cycle from the beginning.

By splitting the model and the view, MVC reduces structural complexity and increases flexibility and usability.

Service

The layer between the controller and the model. It requests data from the model and gives it to the controller. With the help of this layer, data storage (model), data retrieval (service), and data handling (controller) can be separated from each other. Since this layer is not part of the original MVC pattern, its use is not mandatory.

Advantages:

- Concurrent development – Several developers can work simultaneously, separately, on the model, controller, and views.
- High cohesion – With the help of MVC, related functions can be grouped into one controller. The views of a certain model can also be grouped.
- Independence – In the MVC pattern the elements are fundamentally largely independent of each other
- Easily changeable – Since the responsibilities are separated, future developments will be easier
- Several views for one model – Models can have several views
- Testability – since the responsibilities are cleanly separated, the separate elements are more easily testable independently of each other

Disadvantages:

The disadvantages of MVC generally stem from the necessary extra code.

- Code readability – The framework adds new layers to the code, which increases its complexity
- A lot of boilerplate code – Since the program code is broken into 3 parts, one of these will do most of the work, and the other two exist due to satisfying the MVC pattern.
- Harder to learn – The developer has to know several different technologies to use MVC.

4 The role of design patterns, their classification, and the presentation of 2 notable design patterns per category.

In computer science, the general, reusable solutions that can be given to frequently occurring programming tasks are called software design patterns. A design pattern is usually a description of objects and classes cooperating with each other.

Design patterns do not provide a finished design that can be directly coded, although they have sample codes, which, however, have to be filled with code suitable for the given situation. Their purpose is to provide a description or template. They help to formalize the solution.

The patterns usually show connections between classes and objects, but do not concretely specify the final classes or objects. Instead of the abstract classes of the models, in some cases interfaces can also be used, although the design pattern itself does not show them. Some languages contain design patterns built in. Design patterns can be regarded as one level of structured programming between the paradigm and the algorithm.

Most design patterns are developed for an object-oriented environment. Since functional programming is little known and used, only few design patterns are known for that environment yet, for example the monad. Not all of the object-oriented patterns can be, and not all are worth, using here. There are some that have to be modified.

Design patterns gained popularity in the early '90s (1994), when the programmer quartet referred to as the "Gang of Four" or GoF – Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides – published their book Design Patterns, which still serves today as a basis for research into object-oriented programming patterns. The design pattern itself was not defined. The concept itself remained unformalized for years.

This book presents a total of 23 patterns, and sorts them into the following categories:

- creational pattern
- structural pattern
- behavioral pattern

Design patterns according to the GoF definition: descriptions of objects and classes cooperating with each other that, in a customized form, solve some general design problem in a certain context. In software design, infinitely many different solutions can be given for a single problem, but among these few are optimal. Experienced software designers, who have already encountered many similar problems, can easily pull out a solution that is already tried and proven. For beginner developers, however, it is useful if they can take in hand, precisely documented, all the knowledge and experience that they could only achieve with years of work, learn from it, and, with the expressions it introduces, more easily discuss their ideas with each other. Design patterns are such collected experiences, each of which gives a generalized answer to a frequently occurring problem. However, in addition to all this, they have numerous advantages:

- they shorten the time of gaining experience. Design patterns were not invented, but rather the optimal answers given to frequently occurring problems were collected, thereby providing solutions that most developers would, sooner or later, figure out on their own – only possibly much later. Of course, it is not excluded that better, more efficient solutions exist, and it is rare that a single pattern can be applied to a problem exactly as it is described in the books, but it is in any case worth getting to know them, if for no other reason than to acquire some of the authors' way of seeing.
- they shorten the design time. All the patterns are well documented, easily reusable, so once we apply them, there is a good chance that at a similar problem they will come to mind again, together with all their advantages and disadvantages. Thus we can immediately give an efficient, flexible solution and spare ourselves a lot of design and possible redesign. Moreover, the consequences section found after the patterns helps us get a more complete picture of the effects of the application too.
- it gives a common vocabulary into the hands of developers. This makes communication among each other and the documentation of the program easier, since it is easier to discuss the solution of a problem if there is a common basis from which we start or to which the new designs can be compared.
- it makes higher-level programming possible. Since these patterns, due to their widespread nature, have already withstood the test of very many programmers, they presumably contain the optimal solution for the problem.

Design patterns organize the modules and their connections. They are at a lower level than architectural patterns, which characterize the general structure of the entire system.

Several different design patterns exist, for example:

- Algorithm strategy patterns
- High-level strategies that describe how to organize an algorithm for a given architecture.
- Computation design patterns
- Aim at finding the key computations.
- Execution patterns

- Patterns dealing, at the lower level of execution, with addressing, the organization, optimization, synchronization of the execution of tasks.
- Implementation strategy patterns
- In the implementation of the source code they support the organization of the program and the construction of data structures important for parallel programming.
- Structural design patterns
- They deal with the global structure of the application.

The GoF originally grouped the design patterns into three categories: creational patterns, structural patterns, and behavioral patterns, for which they used the concepts of delegation, aggregation, and consultation. In an object-oriented environment they also use the concepts of inheritance, polymorphism, and abstract ancestor class, although abstract classes can mostly also be interfaces. Some authors also distinguish architectural design patterns, such as the model-view-controller.

4.1 Creational patterns (Singleton, Prototype)

4.1.1 Singleton

Ensures that only one instance is created from the class, and provides a public access to this instance.

The singleton design pattern is a design pattern that restricts the number of creatable instances of a class to one object. It is common that a class has to be written so that only one instance of it can exist. For this one has to know well the principles of object-oriented programming. An instance of a class can be created with its constructor. If there is a public constructor in the class, then any number of instances can be created from it, so the singleton cannot have a public constructor. But if there is no constructor, then the instance through which we could call its methods cannot be created. The solution is the class-level (static) methods. These can be called even if there is no instance. The singleton therefore has a class-level method (getInstance), which returns the same instance to every caller. Of course, this instance also has to be created; for this a private constructor has to be made, which the getInstance can call as a member of the singleton class.

4.1.2 Prototype

Determines a preliminary template for creating objects, and this is later copied. We often use it when the exact type of the object turns out only at run time.

The main technique of the prototype design pattern is cloning. The task of cloning is to create an object identical to the original object. Simple assignment is not suitable for this, because it only copies the reference of the object, so the two references point to the same place. There are two kinds of cloning: shallow cloning (shallow copy) and deep cloning (deep copy).

4.2 Structural patterns (Decorator, Proxy)

4.2.1 Decorator

Makes it possible to dynamically add further functions, responsibilities, without changing the abstraction.

The decorator pattern is a classic example of transparent wrapping. Its classic example is the Christmas tree. By the fact that I put a ball on the Christmas tree, it still remains a Christmas tree, that is, the decoration is transparent. We achieve this by having both classes participating in the object composition derive from the same ancestor, that is, they are of the same type. This is useful because the decorating elements often change; it is easily conceivable that a new decoration has to be added. If the decorator were a separate type, then the Christmas-tree-processing algorithms could possibly be complicated.

For the decorator pattern we start from an abstract ancestor. This has two kinds of children: base classes, which can be decorated, and decorator classes. In the Christmas tree example the base classes are the various fir trees. We usually organize the decorator classes under an abstract decorator class, but this is not mandatory.

During decoration every method of the ancestor has to be implemented so that we call the method of the wrapped instance, and where necessary, we add the extra functionality there. We can speak of two kinds of decoration:

- When we extend the responsibility of the existing methods. Such is the Christmas tree example.
- When we also add new methods to the existing ones. Such is the handling of Java streams, and the rentable vehicle example below.

In both cases the instantiation typically happens like this:
AncestorClass instance = new DecoratorN(...new Decorator1(new BaseClass())...);

Since the wrapping is transparent, we can wrap our instance any number of times, even twice with the same decorator. This results in an extremely dynamic, easily extensible structure, which with inheritance could be realized only with very many classes.

It is interesting to observe on the UML figure of the pattern that from the decorator class an aggregation points back to the ancestor class. This resembles the solution of the Employee - Boss relation of database handling, when the Employee table is in a one-to-many relationship with itself, where the foreign key contains the employee_ID value of the boss.

4.2.2 Proxy

A design pattern applied for hiding, replacing another object.

The proxy design pattern gives an example of a very simple composition that is, moreover, transparent wrapping. An instance that is interesting in some respect (expensive, remote, security-sensitive, ...) is owned by its proxy. This interesting object cannot be reached from the outside; its services can be reached only through its proxy. At the same time the outside world thinks that it reaches the interesting object directly, because the proxy wraps the interesting object transparently. Because of the transparency the proxy and the interesting object have a common ancestor.

Many kinds of proxy exist, according to in what respect the replaced object is interesting, e.g.:

- Virtual proxy: Replacement of resource-intensive objects (e.g. an image) by postponing the instantiation (or other expensive operation) as long as possible. Word processors use this for loading images. If I only quickly flip through the document, then the image is not loaded (the loading is postponed), only its place is shown.
- Remote proxy: Local presentation of remote objects in a transparent way. The client does not even perceive that the actual object is on another machine, as long as there is a network connection. Remote method invocation (RMI) applies this.
- Protection proxy: Regulates access in the case of different rights.
- Smart reference: Replaces the simple reference in cases when further operations are needed when accessing the object.
- Cache: If there is a computation (including downloads here) that is expensive, then it is worth storing the result of the computation in a cache, which is also a kind of proxy.

4.3 Behavioral patterns (State, Observer)

4.3.1 State

The behavior of the object can be changed depending on the internal state.

It makes it possible for an object's behavior to change when its state changes. Example: the TCPConnection class represents a network connection; it can have three states: Listening, Established, Closed; it handles the requests depending on its state.

We use it if

- the behavior of the object depends on its state, and it has to change its behavior at run time according to the current state, or
- the operations have large conditional branches that depend on the state of the object.

Advantages:

- It encapsulates the state-dependent behavior, so introducing new states is easy.
- More transparent code (no large switch-case construct).
- The State objects can be shared.

Disadvantages: The number of classes increases (use it only in justified cases).

4.3.2 Observer

Determines a one-to-many dependency between objects. About the modification of a given object it automatically sends notification information to the objects dependent on it, which update based on these.

It makes it possible for an object, in the case of its change, to notify arbitrary other objects without knowing anything about them. Its parts:

- Subject: Stores the registered observers, offers an interface for registering and unregistering observers as well as notifying them.
- Observer: Defines an interface for those objects that want to be informed of the change occurring in the subject. The update method serves this purpose.

We know two kinds of observer realization:

- "Pull" observer: In this case the observer pulls the changes from the subject.
- "Push" observer: In this case the subject pushes the changes to the observer.

The difference between the two lies in what parameter the update method receives. If the subject passes itself (with the help of an update(this) call) to the observer, then through this reference the observer is able to query the changes. That is, this is the "pull" solution.

If the subject passes to the update method those fields of it that have changed and that the observer observes, then we speak of the "push" solution. In the following example we can see exactly such a realization.